Algoritmi e Strutture Dati

IV Appello

20 Settembre 2023

Commento agli esercizi proposti nella traccia

A seguire trovate una serie di commenti ai vari esercizi proposti nella traccia d'esame. Non si tratta di una soluzione agli esercizi, quanto di una collezione di osservazioni legate ai contenuti e al modo più idoneo di affrontare le varie richieste.

1 12pt - Semplificato

1.1 E1

Si tratta di un classico problema di iterazione vincolata su una struttura dati bi-dimensionale.

Per ogni posizione (i, j) occorre effettuare somme di elementi, per un numero di passi pari a k e lungo 4 direzioni diverse. Occorre tenere conto del ruolo dei bordi della matrice e assicurarsi di non sforare i bordi stessi, per cui indipendentemente dal modo con cui si decida di procedere, bisogna assicurarsi che l'indice di riferimento sia sempre nel range $0 \leq \ldots < r$ per le righe ($0 \leq \ldots < c$ per le colonne).

Poiché si chiede che la matrice sia allocata dentro alla funzione, e che i risultati siano disponibili al chiamante, il prototipo va completato introducendo una matrice destinazione passata dal chiamante *per riferimento*, in modo che l'allocazione sia visibile anche da fuori.

1.2 E2

La definizione dei tipi dedicati a lista (`link` head) e nodo (char `c`, `link next`, `link` prev) è praticamente standard, non sono caratterizzati da nulla di particolare rispetto a contenuti visti a lezione. Si applicano direttamente le definizioni di ADT e quasi ADT per decidere dove posizionare i contenuti.

La funzione di compattazione ha bisogno di fare una scansione lineare della lista, per individuare eventuali punti di cancellazione.

Per come è formulato il problema, non è mai necessario effettuare una cancellazione in testa, poiché l'eventuale primo nodo di una qualsiasi sequenza è sempre preservato. Di contro, la cancellazione in coda è possibile, e quindi occorre tenerne conto come caso particolare, visto la necessità di aggiornare anche i puntatori prev dei nodi, e tentare un accesso next->prev sulla coda non sarebbe ammissibile.

Il modo più immediato per implementare correttamente la funzione richiesta sarebbe quello di prevedere un ciclo esterno di iterazione sulla lista, e, per ogni nodo, un ciclo interno di iterazione per valutare la lunghezza della sequenza di nodi uguali per valore, ed eventualmente un secondo ciclo di cancellazione. Necessario ricordare che non si vuole fare *memory leak*, per cui non è sufficiente

aggiornare i puntatori ma si è tenuti anche a liberare la memoria associata ai nodi. Anche in seguito a cancellazione, poiché non andiamo a toccare direttamente il nodo corrente, il ciclo di iterazione più esterno non è toccato direttamente, e può poi procedere con il passo successivo. Il caso estremo è una cancellazione in coda, che comunque risulta in un avanzamento dell'iteratore verso un nodo NULL, che idealmente coincide con la condizione di stop per il ciclo stesso, che termina correttamente.

1.3 E3

Dato un multi-insieme S, ovvero un insieme per cui siano ammessi duplicati e quindi per cui si vadano a distinguere le singole istanze di elementi con il medesimo valore, si vogliono identificare due possibili sotto-(multi)insiemi di S stesso, S_1 e S_2 tali per cui la somma degli elementi del primo e del secondo differisca esattamente di un valore pari a una costante s, parametro del problema.

Occorre prestare particolare attenzione al fatto che il testo NON chiede che siano usati necessariamente tutti gli elementi di S. La condizione di ottimalità chiede di massimizzare il numero di elementi usati, ma non è detto che sia possibile e/o necessario usarli tutti. Come conseguenza, non è corretto tentare di risolvere il problema come un partizionamento a due vie, poiché è necessario essere disposti a scartare alcuni elementi, e quindi è necessario prevedere un terzo insieme, di scarti appunto, in cui accumulare gli elementi non scelti né per S_1 né per S_2.

Il modo più diretto per ottenere tale risultato sarebbe procedere costrunendo una soluzione rappresentata mediante un vettore di interi, lunga quanto gli elementi di S, a cui si assegna un valore tra 3 possibili in ogni posizione (non preso, primo insieme, secondo insieme). Tale soluzione si può leggere come *l'elemento di posizione i è associato all'insieme sol*[i].

Non c'è motivo di scomodare modelli multipli e tentare di costruire prima S_1 e poi S_2 con i suoi scarti, poiché sarebbe necessario far partire una visita ricorsiva dentro alla condizione di terminazione della prima.

A fronte di soluzioni già note, volendo si può applicare del pruning tenendo conto della cardinalità corrente dei due insiemi generati, e del numero di elementi ancora liberi (non scartati). Se non ne rimangono abbastanza per superare la soluzione ottima trovata sino ad ora, è possibile abbandonare la strada corrente.

2 18pt - Completo

2.1 Contesto

Si fa riferimento a quello che in letteratura prende il nome di *k-capacitated tree partition*, di cui si riporta nuovamente la definizione per comodità. Dato un grafo (pesato non orientato, in questo caso) $G = (V, E)$, si definisce tale una collezione di sotto-insiemi di archi $\{E_1, E_2, \ldots, E_n\}$ con $E_i \subseteq E$, disgiunti per vertici (un certo vertice può apparire come estremo di più archi, ma questi devono appartenere al medesimo E_i), tali per cui ogni E_i induce un albero di almeno k nodi. Si aggiunge poi un vincolo legato all'ottimalità della soluzione, minimizzare la somma complessiva del peso degli archi scelti, che però non incide sulla natura del *tree partition*.

Il punto fondamentale, prima di procedere con la discussione delle singole richieste, è sottolineare come si abbia di fronte un problema guidato dal punto di vista degli archi e non dei nodi.

2.2 Strutture dati

È sufficiente fare uso della versione più semplice della struttura grafo, senza bisogno di introdurre tabelle di simboli per i vertici. La versione a matrice di adiacenza, sebbene meno efficiente in

memoria, in questo caso può essere più agevole per l'uso che se ne intende fare. Una rappresentazione a liste è comunque assolutamente ragionevole.

Per quanto riguarda la struttura dati con cui rappresentare un tree partition, può essere sufficiente prevedere una collezione di liste/vettori di archi, o di indici di archi nel caso in cui si sia materializzata la collezione di archi del grafo originale, per tenere traccia dei contenuti dei vari sotto-insiemi. Volendo si possono rappresentare esplicitamente i grafi indotti dai singoli sotto-insiemi, e quindi fare uso di una collezione di grafi, ma non è strettamente necessario spingersi a tanto.

2.3 Verifica

Per quanto riguarda la proposta di soluzione, non bisogna perdere di vista il concetto di base: un tree partition è una collezione di insiemi di archi. In quanto tale, ci si aspetta una proposta di soluzione che sia interpretabile come tale. Fornire una proposta come elenchi di vertici, per indicare quali vertici fanno parte di una certa partizione, non permette di fornire una soluzione completa. Dati i vertici di un insieme, si è persa l'informazione circa quali archi facciano parte, o meno, di tale sotto-insieme. Data una proposta di soluzione, gli aspetti da verificare sono:

- disgiunzione per vertici: un vertice può essere toccato da più archi, ma questi devono apparire nel medesimo sotto-insieme
- il sotto-grafo indotto è un albero: occorre verificare <u>connessione</u> e <u>aciclicità</u>
- cardinalità minima: il sotto-grafo indotto deve avere *almeno k* vertici

2.4 Ottimizzazione

La ricerca di una soluzione ottima può essere vista come un problema di assegnazione di archi a sotto-insiemi distinti, con anche la possibilità di scartarne alcuni. Si può introdurre un sotto-insieme fittizio di scarti, dove accumulare tutti quelli non scelti. Il numero di sotto-insiemi non è noto a priori, ma dato il vincolo sulla cardinalità, sappiamo che è vincolato superiormente da $\lceil \frac{|V|}{k} \rceil$ (più lo scarto).

Solo per questo punto, volendo si sarebbe potuto affrontare il problema partizionando l'insieme di vertici (senza scarti), e aggiungere il calcolo di un *minimum spanning tree* per ognuno dei potenziali sotto-insiemi, e riportarsi al formalismo originale di tree partition con selezione degli archi effettivi che si siano inclusi nella soluzione corrente. Fermarsi a un partizionamento dei vertici, non porta a una soluzione prendendo in considerazione il contesto del problema, come discusso al punto 2.3.